

RELATÓRIO DE DESENVOLVIMENTO: APRENDIZADO POR REFORÇO PROFUNDO APLICADO AO AMBIENTE MS. PAC-MAN (ATARI 2600)

Disciplina / Contexto: Projeto de Inteligência Artificial / Engenharia de Computação

Autor: [Felipe de Medeiros, Felipe Samuel e Thiago Augusto]

Data: Junho de 2026

1. INTRODUÇÃO E OBJETIVO

O objetivo deste projeto foi o desenvolvimento, treinamento e validação de um agente inteligente capaz de maximizar a pontuação e mitigar falhas mecânicas de movimentação no jogo Ms. Pac-Man (versão [ALE/MsPacman-v5](#) do framework Gymnasium).

O projeto utilizou a técnica de **Aprendizado por Reforço Profundo (Deep Reinforcement Learning)**, evoluindo de uma abordagem clássica puramente computacional-visual até uma arquitetura de vanguarda híbrida de múltiplas entradas (Multi-Input). A infraestrutura de execução foi integrada ao ecossistema **LangChain** para garantir modularidade, encapsulamento e padrões industriais de engenharia de IA.

2. FUNDAMENTAÇÃO TEÓRICA: A EVOLUÇÃO DO ALGORITMO DQN

Para compreender o salto de desempenho obtido nas eras avançadas deste projeto, é necessário analisar a lógica estrutural e matemática que diferencia as três variações algorítmicas avaliadas: o **DQN Tradicional**, o **Double DQN (DDQN)** e o **Dueling DQN**.

2.1 DQN Tradicional (Deep Q-Network)

O DQN original substitui as tabelas de correspondência do Q-Learning clássico por uma Rede Neural Convolucional (CNN) para aproximar a função de valor $Q(s, a)$. A rede recebe o estado s (pixels da tela) e prevê o retorno esperado para cada ação. O algoritmo minimiza o erro da Equação de Bellman através do Erro Quadrático Médio (MSE).

- **Gargalo de Superestimação:** O DQN tradicional sofre de um viés de superestimação sistemática dos Q-Values. Como a fórmula de atualização utiliza o operador de máximo ($\max_a Q(s', a)$) tanto para **selecionar** a melhor ação quanto para **avaliar** essa ação, a rede infla artificialmente os valores. Se a rede superestimar o valor de uma rota de

fuga ruim devido a um ruído visual (como uma quina do labirinto), o agente insistirá no erro, gerando platôs de desempenho.

2.2 Double DQN (DDQN) — Desacoplamento de Seleção e Avaliação

O Double DQN altera a formulação matemática de atualização para mitigar o viés de superestimação, introduzindo duas redes neurais com pesos distintos: a **Rede Online** (theta) e a **Rede Alvo/Target** (theta').

- **Mecânica:** A Rede Online é utilizada exclusivamente para **selecionar** a melhor ação do próximo estado (s'), determinando qual escolha maximiza o valor estimado. Em seguida, a Rede Target é acionada para **avaliar** o Q-Value real daquela ação específica selecionada. Ao retirar da mesma rede a dupla função de escolha e julgamento, os gradientes tornam-se estáveis, impedindo que o agente crie expectativas falsas sobre corredores sem saída.

2.3 Dueling DQN — Divisão Arquitetural do Grafo Computacional

Diferente do DDQN, que modifica a função de perda, o Dueling DQN altera a própria anatomia interna da rede neural. Após as camadas convolucionais, o fluxo de tensores é dividido em dois canais analíticos paralelos:

- **Fluxo de Valor $V(s)$:** Uma camada densa que calcula o valor intrínseco de estar no estado atual s , independentemente da ação tomada (ex: mensurar quão perigoso é estar em um cruzamento específico mapeado).
- **Fluxo de Vantagem $A(s, a)$:** Uma camada densa que calcula o impacto de escolher cada ação específica a em relação às outras alternativas.

A fusão das sub-redes ocorre na camada de saída, subtraindo a média das vantagens para garantir a identificabilidade matemática da equação:

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|A|} \sum_{a'} A(s, a') \right)$$

Esta abordagem acelera o aprendizado em ambientes como Ms. Pac-Man, pois permite à rede entender o valor global do cenário mesmo quando as ações imediatas são irrelevantes (como correr por um longo corredor linear).

3. A JORNADA DE EVOLUÇÃO: OS 3 MODELOS TREINADOS

3.1 MODELO 1: Dueling Double DQN Clássico (A Era Visual)

O primeiro modelo baseou-se estritamente na percepção visual do ambiente. O agente recebia apenas os pixels brutos do emulador, processados por camadas convolucionais.

- **Configuração:** Integração do Dueling DQN com Double DQN (Dueling DDQN)

operando sobre a imagem convertida para escala de cinza, redimensionada para 84×84 e empilhada em 4 frames (`state_stacked`).

- **Gargalo Encontrado:** O agente atingiu um platô de pontuação e desenvolveu o comportamento crônico de **trancamento nas quinas e paredes**. A compressão e redução de resolução da imagem geravam "pontos cegos", impossibilitando a diferenciação visual de quinas seguras versus quinas bloqueadas sob estresse (perseguição de fantasmas).

[INSERIR AQUI OS DADOS DO MODELO 1: Gráfico de Treino, Hiperparâmetros Iniciais ou Notas do Modelo 1]

3.2 MODELO 2: Otimização de Hiperparâmetros e Reward Shaping (A Era do Fluxo)

Para forçar o agente a quebrar o platô e punir o comportamento de inércia nas paredes, o segundo modelo focou em engenharia de recompensa (Reward Shaping) e correções críticas na memória de repetição.

- **Modificações de Recompensa:** Injeção de penalidade de -0.1 para frames ociosos, bônus agressivo de +15.0 para manutenção de sobrevivência em scores altos e punição severa de -50.0 para perda de vidas.
- **Corretivo do Bug do Replay Buffer:** Correção no loop principal onde ações aleatórias forçadas para destravar o agente eram registradas incorretamente no buffer. O fluxo foi invertido para garantir que a ação real enviada ao `env.step()` fosse honestamente computada pelo gradiente.
- **Efeito:** O modelo alcançou picos excelentes de pontuação, mas a sensibilidade visual continuou instável em cenários de alta densidade de fantasmas.

[INSERIR AQUI OS DADOS DO MODELO 2: Gráfico de Treino ou Configuração do Seu Reward Shaping]

3.3 MODELO 3: Arquitetura Híbrida Multi-Input (O Plano B Atual)

A solução definitiva para o problema de trancamento geométrico consistiu em abandonar a dependência de imagem pura, transformando o agente em uma estrutura híbrida.

- **Extração de Baixo Nível (RAM Peek):** Mapeamento direto da memória volátil do Atari para obter as coordenadas cartesianas exatas (X,Y) da Ms. Pac-Man e as posições horizontais dos 4 fantasmas (Blinky, Pinky, Inky e Sue), normalizadas entre 0.0 e 1.0.
- **Camada de Fusão Tensa (`Concatenate`):** Expansão do grafo computacional do TensorFlow para aceitar duas entradas simultâneas (`image_input` e `feature_input`). O vetor da imagem linearizado pelo `Flatten()` é fundido ao vetor de tamanho 6 da RAM.
- **Transfer Learning Cirúrgico:** Aplicação de transplante de pesos. As camadas `Conv2D` do modelo anterior (que já sabiam identificar objetos) foram congeladas e copiadas para a nova rede, permitindo que os episódios subsequentes treinassem exclusivamente a

leitura da "bússola geométrica" da RAM pelas camadas **Dense**.

4. INFRAESTRUTURA DE IA E ARQUIVOS DE PRODUÇÃO

Para manter a elegância do documento e a modularidade do projeto, o código-fonte de implementação do pipeline híbrido (arquivo **play.py**) e a esteira de conexões do **LangChain** via **RunnableLambda** (O **RunnableLambda** é um dos componentes fundamentais do **LangChain Expression Language (LCEL)**). Ele serve para transformar qualquer função Python comum em um objeto executável que pode ser encadeado diretamente em uma *chain* (corrente) usando o operador pipe (`|`).

Quando você constrói fluxos no LangChain, os componentes nativos (como prompts e LLMs) já herdam a classe **Runnable**. No entanto, se você precisar injetar uma lógica customizada sua no meio do caminho — como limpar um texto, formatar um dicionário ou filtrar dados —, você precisa envelopar sua função com o **RunnableLambda**.)

foram completamente isolados deste relatório.

- **Código de Produção:** Fornecido separadamente no arquivo digital anexo **play.py**.
- **Pesos do Modelo Híbrido:** Arquivo de checkpoints calibrados **dueling_ddqn_mspacman_hybrid.weights.h5**.

O código em anexo apresenta a função de extração de telemetria da RAM por endereço de registradores e a inferência em tempo real com taxa de exploração zerada (Epsilon = 0) para garantir a execução puramente cognitiva do agente.

5. ANÁLISE DE RESULTADOS E MÉTRICAS COMPARATIVAS

Abaixo é apresentada a tabela consolidada de desempenho dos três paradigmas desenvolvidos ao longo do projeto.

Métrica	M1: Visual Puro	M2: Reward Shaped	M3: Híbrido (Plano B)
Média Frequente	300 a 500 pts	600 a 900 pts	1.100 a 1.800 pts
Pico Máximo	4.300 pts (<i>Instável</i>)	2.600 pts	4.010 pts (<i>Estável</i>)
Pico Secundário	-	-	3.070 pts

Análise Crítica do Fechamento do Modelo 3

O Modelo 3 teve seu treinamento finalizado no episódio **13.226** devido ao esgotamento das cotas de Unidade de Processamento Gráfico (GPU) em nuvem. A transição forçada para Unidade Central de Processamento (CPU) inviabilizou o tempo hábil de cálculo de gradientes por frame. No entanto, conforme demonstrado pelas métricas, a janela entre os episódios 12.950 e 13.200 foi matematicamente suficiente para o otimizador Adam convergir os pesos da camada de fusão, atingindo o ápice estável de **4.010 pontos no episódio 13.137**.

6. CONCLUSÃO

A transição de um paradigma puramente focado em Visão Computacional para um ecossistema Híbrido provou ser a estratégia correta para contornar o problema de platô de relevância espacial em Reinforcement Learning Profundo. O agente atual demonstra mapeamento espacial preditivo, utilizando as variáveis escaladas da RAM para recalcular rotas de fuga antes que a colisão visual iminente ocorra. Como trabalhos futuros, sugere-se a adição de técnicas de Prioritized Experience Replay (PER) para acelerar a ejeção de memórias antigas e espúrias do buffer de repetição.